Bienvenida al programa: ruta, gates y diagnóstico técnico (Bloque 1) / El punto de llegada: qué vas a tener construido al terminar el programa

✓ **Hecho:** Ver

📘 **UNIDAD U1.1** · Alineación estratégica y perfil de egreso

El punto de llegada: qué vas a tener construido al terminar el programa

Ya viste el mapa completo del programa: **3 módulos, 3 Gates, un posible cinturón al cierre**. Ahora conviene bajar eso a algo concreto.

👉 ¿Qué es exactamente lo que vas a tener funcionando cuando termine el programa?

Esta lección describe esa meta con el mayor detalle posible, aunque todavía no exista nada construido, para que sepas hacia dónde apunta cada semana de trabajo.

🧠 Prerrequisitos y entorno mental

Antes de seguir, vale la pena aclarar qué se asume que ya sabes y qué no.

➡️ El programa asume **Python básico** (variables, funciones, estructuras de control): no necesitas haber trabajado antes con **APIs**, ni dominar **Git** o la terminal de antemano, esas piezas se instalan recién en el **Módulo 1**, desde cero.

➡️ Tampoco necesitas saber ya qué es **Docker**, **FastAPI**, **LangChain** o **RAG**: son exactamente lo que vas a construir a lo largo del programa, no lo que se te exige traer puesto.

Si en esta lección algún término técnico te suena lejano, es normal: cada uno tiene su propia unidad

dedicada más adelante ("**Cómo cada módulo aporta una capa**", más abajo, muestra en qué momento aparece cada uno).

Esta lección solo te muestra hacia dónde apunta el recorrido completo, no lo que tienes que saber hoy.

🚀 La aplicación que vas a entregar en el Gate 3

El **Gate 3** pide una **aplicación generativa end-to-end** sobre un problema de **negocio real**, elegido según tu **propio perfil**.

No importa el dominio exacto que elijas (atención al cliente, búsqueda interna de documentos, generación de reportes, soporte técnico, o cualquier otro caso de uso razonable), la anatomía técnica final es la misma para todos:

> ***"un servicio expuesto por API, con orquestación de pasos, capacidad de responder con información recuperada de una fuente propia (RAG), empaquetado en un contenedor y documentado para que otra persona pueda instalarlo y ejecutarlo sin tu ayuda."***



💡 **Piensa en un ejemplo concreto para fijar la idea**:

➡️ Un asistente que responde preguntas sobre las políticas internas de una empresa.

➡️ Una persona usuaria escribe su pregunta en lenguaje natural.

➡️ La aplicación busca los fragmentos relevantes en los documentos indexados de la empresa.

➡️ Arma una respuesta fundamentada citando de dónde la sacó.

➡️ Y la entrega en segundos a través de un endpoint HTTP.

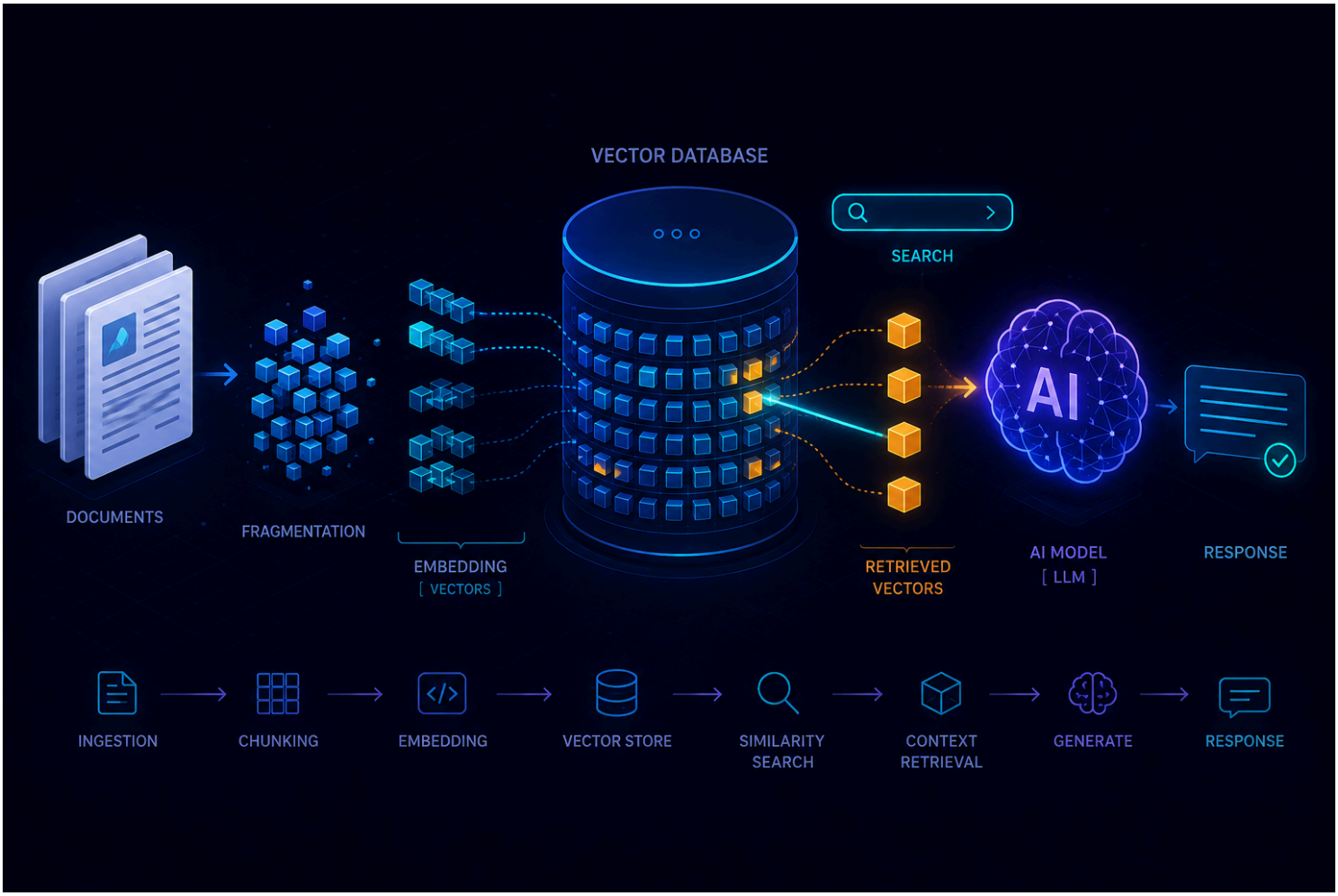
Este es solo un ejemplo para imaginar el proceso. Tu proyecto puede abordar un tema diferente y adaptar esta estructura según lo que necesites.

Vale la pena detenerse en qué pasa exactamente detrás de esa respuesta, porque cada paso corresponde a algo puntual que vas a construir más adelante. Primero, todos los documentos de política interna se dividen en fragmentos manejables (chunking) y cada fragmento se transforma en un vector numérico que representa su significado (embedding); esos vectores quedan guardados en una base de datos vectorial.

Cuando llega la pregunta, la aplicación la transforma también en un vector, busca los fragmentos más parecidos en significado (no en palabras exactas) y se los entrega al modelo como contexto.

El modelo genera la respuesta apoyándose en ese contexto, no en lo que "recuerda" de su entrenamiento, y la aplicación registra de qué fragmento salió cada dato para poder citarlo.

🔎 **Ese mecanismo completo tiene nombre**: es exactamente lo que en el programa llamamos **RAG** (retrieval-augmented generation), y es una de las piezas que vas a construir en el Módulo 2.



💭 **Ahora imagina la misma pregunta pero con un caso más complejo**: ¿Cuántos días de vacaciones me quedan y cómo los solicito?.

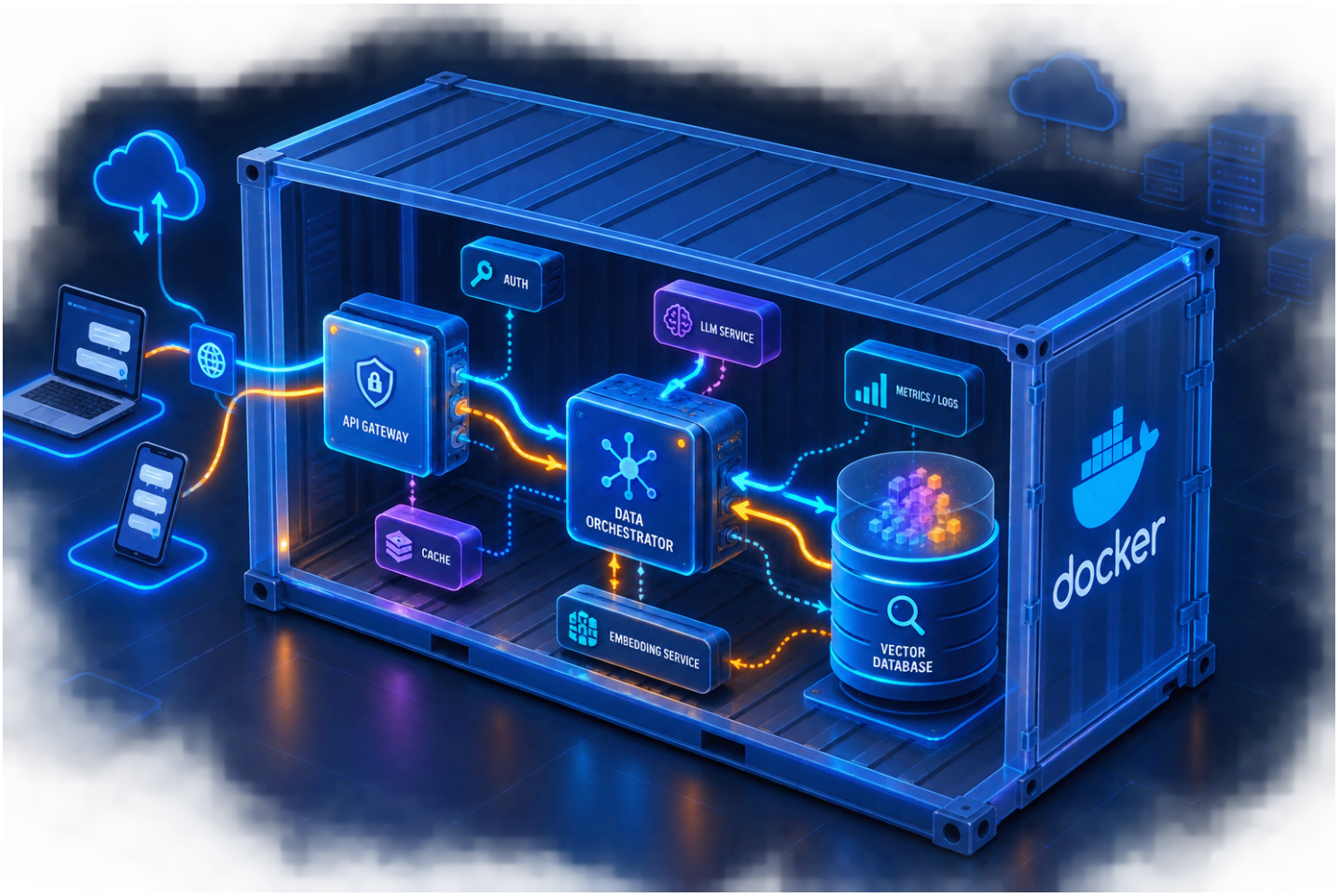
Responder bien exige **combinar dos cosas**:

➡️ buscar el reglamento de vacaciones (RAG)

➡️ ejecutar una acción o consultar un dato externo (por ejemplo, un sistema de recursos humanos).

Ese segundo tipo de capacidad, decidir qué herramienta usar y en qué orden, es lo que en el programa se construye como un agente.

🔗 **Tu aplicación final probablemente combine ambos mecanismos: RAG para el conocimiento, y un agente para las acciones.**



Cómo cada módulo aporta una capa

Ese resultado final no aparece de golpe en la ultima semana de clases: se construye por capas, una por módulo.

Del Módulo 1 sale la primera capa

Un programa que consume un modelo de lenguaje vía **API**, procesa la respuesta en formato estructurado y maneja errores del proveedor.

Ahí también queda definida la **arquitectura general** de la aplicación (qué componentes va a tener y cómo se van a comunicar) y el primer banco de prompts de sistema, probado para que produzca salidas consistentes. Es simple, corre en tu máquina, y ya resuelve el problema de fondo: conversar con un modelo de forma programática y confiable.

Del Módulo 2 sale la capa más grande

Porque es el módulo con más horas del programa. Ese programa simple se convierte en un **cliente modular** con manejo de límites de uso y costo, gana un pipeline de procesamiento de texto con métricas de evaluación, se transforma en un flujo orquestado de varios pasos con **LangChain**, incorpora un agente con herramientas propias y memoria de contexto, y aprende a indexar tu propio **corpus** y responder con **RAG**.

Cierra con observabilidad y seguridad: logging, versionado de prompts y una matriz de riesgos con mitigaciones. Al terminar este módulo tu proyecto ya "sabe" cosas que no vienen del modelo, sino de tus propios documentos, y queda registro de cómo se comporta.

Del Módulo 3 sale la capa que lo convierte en un producto real

Se expone como servicio con **FastAPI** con validación tipada, se empaqueta en un contenedor con **Docker** y se compone con otros servicios, se integra todo de punta a punta con al menos una ejecución completa documentada, se somete a una **Matriz de Pruebas** con casos válidos, inválidos y de borde, y se documenta con un **README** técnico que permite a otra persona instalarlo y ejecutarlo sin preguntarte nada.

🎯 **El Gate 3 evalúa exactamente esa suma de capas funcionando juntas, no una capa aislada**: de nada sirve un RAG impecable si el servicio no levanta en un contenedor limpio, y de nada sirve un contenedor perfecto si adentro no hay nada que responda correctamente.

📦 Lo que entregas físicamente en el Gate 3

Cuando llegue esa última semana, la entrega no es solo "una app que funciona". Son piezas concretas y verificables, todas dentro de un mismo repositorio:

El código y el contenedor

El repositorio completo, con una imagen de **Docker** que cualquiera puede levantar con un par de comandos, sin depender de tu computador ni de configuración manual.

El README técnico

Instalación, ejecución, ejemplos reales de uso de cada endpoint, y las decisiones de arquitectura que tomaste (y por qué las tomaste).

La Matriz de Pruebas y la Matriz de Riesgos

Evidencia de que probaste tu propia aplicación con casos válidos, inválidos y de borde, y que identificaste y mitigaste riesgos como alucinaciones, fuga de datos o **inyección de prompt**.

La demo en vivo

Una demostración corta y sin preparación de guion memorizado: alguien del equipo evaluador puede pedirte que muestres cualquier parte de la aplicación funcionando en el momento.

La presentación ejecutiva

Una defensa breve del proyecto orientada a negocio: qué problema resuelve, qué arquitectura elegiste, qué riesgos identificaste y cómo los mitigaste.



🎤 Un recorrido por el día de la defensa

Para que la meta se sienta todavía más real, esto es aproximadamente lo que vas a vivir en la semana del **Gate 3**.

1

📦 Entrega de la versión final

Antes de tu turno, entregas tu repositorio con la etiqueta o rama que corresponde a tu versión final: esa es la que se evalúa, no lo que sigas cambiando después.

2

🦾 Levantamiento de la aplicación

En la sesión, primero levantas tu aplicación desde el contenedor frente al equipo evaluador, sin pasos manuales adicionales ni configuración de último minuto.

3

📹 Demo en vivo

Luego haces la **demo** en vivo: no una grabación ni una secuencia de clics ensayada, sino la aplicación respondiendo preguntas reales que te van proponiendo en el momento, incluyendo alguna pensada para provocar un caso límite.

4

👴💼 Presentación ejecutiva

Después de la **demo** viene la presentación ejecutiva, donde explicas el problema de negocio que elegiste, por qué tomaste las decisiones de arquitectura que tomaste, qué riesgos identificaste en tu **Matriz de Riesgos** y cómo los mitigaste.

5

🔎 Preguntas y revisión técnica

El equipo evaluador puede pedirte que muestres una parte específica del código, que expliques una decisión puntual, o que ejecutes un endpoint distinto al que mostraste.

⚠️ **La rúbrica calificada de 8 dimensiones evalúa todo esto en conjunto, y recuerda que tiene criterios críticos no compensables**: si el servicio no corre en el contenedor de forma reproducible, por ejemplo, ese criterio no se compensa con una presentación ejecutiva brillante.

🚫 Qué no es este proyecto

Para calibrar bien el nivel de exigencia, es útil decir también qué es lo que este proyecto no es.

> **No es un prototipo que solo funciona con la pregunta exacta que usaste para probarlo**: tiene que sobrevivir a las tres Matrices de Pruebas con casos que tú no elegiste para lucirte.

> **No es una demo grabada de antemano por si algo falla en vivo**: se evalúa corriendo, en tiempo real, sobre tu propio contenedor.

> **Y no es un ejercicio de un solo módulo disfrazado de proyecto final**: un flujo de RAG impecable sin **API expuesta**, o una **API prolija** sin ninguna capacidad generativa real detrás, no cumplen con lo que pide el **Gate 3**.

❓ Preguntas frecuentes sobre el Gate 3

Antes de seguir, algunas dudas típicas que evitan falsas expectativas desde el primer día:



¿Necesito saber Docker o FastAPI desde ya?

No. Se enseñan recién en el Módulo 3. Para empezar el Módulo 1 alcanza con Python básico.



¿Tengo que usar el ejemplo del asistente de políticas internas?

No. Es solo un ejemplo para fijar la idea; tu proyecto puede abordar cualquier dominio de negocio razonable, elegido según tu propio perfil.



¿Puedo aprobar el Gate 3 con un RAG muy bueno aunque no exponga la API o no tenga contenedor?

No. El Gate evalúa la suma completa (RAG y/o agente, API expuesta, contenedor, documentación) funcionando junta, no una capa aislada por buena que sea.

> 🎥 **¿Puedo preparar una demo grabada por si algo falla en vivo?**
>
> No. La demo se hace en vivo sobre tu propio contenedor, respondiendo preguntas que no elegiste tú.

🧭 Por qué conviene tener esto claro desde el primer día

Cada unidad del programa, desde la siguiente hasta la última, agrega una pieza de esta lista.

> Cuando en la **Unidad 1.3** construyas tu primer módulo Python de consumo de servicios, ya estás construyendo una parte del código que vas a entregar en el **Gate 3**.

> Cuando en la **Unidad 2.1** desarrolles tu cliente modular con manejo de límites y costo, y en la **Unidad 2.5** indexes tu primer **corpus** en una base vectorial, ya estás construyendo la capacidad de **RAG** que tu aplicación final necesita.

> Cuando en la **Unidad 3.2** expongas tus primeros **endpoints** con **FastAPI** y en la **Unidad 3.3** los contenedores con **Docker**, ya estás armando la forma final en la que tu proyecto se va a entregar.

💡 **Nada de lo que hagas en el camino es un ejercicio aislado**: todo se acumula hacia esta misma meta.

📝 Antes de seguir: checklist rápido

Antes de pasar a completar tu Ficha de Alineación Técnica, confirma que tienes claro lo siguiente:

> ✓ Mi proyecto final es un servicio expuesto por **API**, no un script que solo corre en mi computador.

> ✓ "Funciona en mi compu" no alcanza: se evalúa corriendo desde el contenedor, con preguntas que no elegí yo.

> ✓ Cada módulo agrega una capa (**Módulo 1**: consumo de API y manejo de errores; **Módulo 2**:

✓ Cada módulo agrega una capa (**Módulo 1**: consumo de API y manejo de errores, **Módulo 2**: orquestación, RAG y/o agente; **Módulo 3**: exposición, contenedor y documentación), y necesito las tres para llegar al Gate 3.

✓ El Gate 3 no es solo código: también entrego **README** técnico, Matriz de Pruebas, Matriz de Riesgos, demo en vivo y presentación ejecutiva.

Siguiente actividad

Bloque 1 · Ficha de Alineación Técnica

Definir tu ruta → **Completar tu ficha**

Con esta imagen del punto de llegada ya en mente, el siguiente paso de esta clase es concreto: definir tu ruta de trabajo y completar tu **Ficha de Alineación Técnica**.

Última modificación: jueves, 24 de septiembre de 2026, 10:09